Information Retrieval Ranking Models

Lecture 5: Sparse Ranking Methods (BM25)

Avishek Anand

Information Retrieval

February 11, 2026

Outline

1. From Vector Space to Probabilistic Ranking
2. BM25: The Probabilistic Standard
3. Query Expansion and Pseudo Relevance Feedback
4. Statistical Language Models for Ranking
5. Hands-On with PyTerrier
6. Connections to Future Lectures
7. Summary and Assignment

Recap: What We Know

Vector Space Model

- Documents and queries as vectors in term space
- Similarity measured by cosine distance
- Foundation for understanding term-based retrieval

Limitations

- All terms weighted equally (no notion of importance)
- Binary term presence doesn't capture frequency
- No principled scoring function

The Ranking Problem

Formal Definition

Given:

- Query q (sequence of terms)
- Document collection D
- Inverted index

Return: Ranked list scored by relevance $P(\text{relevant}|q, d)$

Key Insight

Not all term matches are equal:

- Rare terms ("syzygy") more informative than common ("the")
- Term frequency matters, but with diminishing returns
- Document length affects scoring

Term Weighting: TF-IDF

Term Frequency (TF)

- More occurrences $\rightarrow$ more important
- $\text{tf}(t, d) = \text{count}(t, d)$

Inverse Document Frequency (IDF)

- Rare terms $\rightarrow$ more discriminative
- $\text{idf}(t) = \log \frac{N}{\text{df}_t}$

Combined Score

$$\text{TF-IDF}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

Problem

Linear TF growth unrealistic:

- 100 occurrences $\neq$ 100$\times$ more relevant than 1
- Need saturation

TF-IDF Example

Query: "information retrieval"

Term	Doc 1	Doc 2	df	IDF	Weight
information	3	1	5000	1.30	High TF
retrieval	2	0	500	2.30	High IDF
the	20	15	50000	0.30	Low IDF

Observation

- "retrieval" gets high weight (rare term)
- "the" downweighted despite frequency (common term)
- But TF still grows linearly $\rightarrow$ need saturation

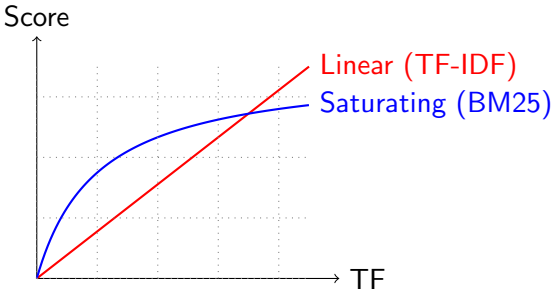
Motivation: TF Saturation

Observation:

- Relevance increases with TF
- But saturates quickly

Example:

- "algorithm" 5× vs. 50×
- 5→10 is meaningful
- 45→50 barely noticeable



Need

Non-linear TF component that saturates gracefully

BM25 Formula

Complete Formula

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

Three components:

1. **IDF:** Term importance (like TF-IDF)
2. **Saturating TF:** Non-linear term frequency
3. **Length normalization:** Penalize long documents

Standard parameters: $k_1 = 1.2$, $b = 0.75$

BM25 Component 1: IDF

IDF Formula

$$\text{IDF}(t) = \log \frac{N - \text{df}_t + 0.5}{\text{df}_t + 0.5}$$

where:

- N = total number of documents
- df_t = document frequency of term t

Properties:

- Similar to TF-IDF but probabilistically motivated
- Can be negative for very common terms ($\text{df} > N/2$)
- Rare terms get high positive IDF
- Common terms get low or negative IDF

BM25 Component 2: Saturating TF

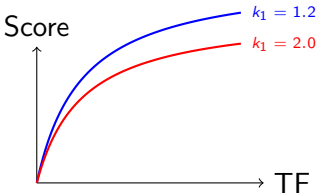
Saturating TF Formula

$$\frac{f(t,d) \cdot (k_1 + 1)}{f(t,d) + k_1}$$

Parameter k_1 (typically 1.2–2.0):

- $k_1 \to 0$: Boolean matching (TF doesn't matter)
- $k_1 \to \infty$: Linear TF (like TF-IDF)
- $k_1 = 1.2$: Moderate saturation (standard)

TF	$k_1 = 1.2$	$k_1 = 2.0$
1	0.55	0.67
5	1.53	1.67
10	1.79	1.83
50	1.98	1.96



BM25 Component 3: Length Normalization

Length Normalization Term

$$1 - b + b \cdot \frac{|d|}{\text{avgdl}}$$

Parameter b (typically 0.75):

- $b = 0$: No length normalization
- $b = 1$: Full normalization (penalize long docs heavily)
- $b = 0.75$: Partial normalization (standard)

Why needed:

- Long documents have more terms just by chance
- Should we penalize or not? Depends on collection
- b controls the trade-off

BM25 Example: Effect of Document Length

Scenario: Term appears 20 times, $k_1 = 1.2$, $b = 0.75$, avgdl=1000

Short Document (500 words)

$$\begin{aligned} \text{norm} &= 1 - 0.75 + 0.75 \cdot \frac{500}{1000} \\ &= 0.625 \\ \text{TF component} &= \frac{20 \cdot 2.2}{20 + 1.2 \cdot 0.625} \\ &= \frac{44}{20.75} = 2.12 \end{aligned}$$

Long Document (2000 words)

$$\begin{aligned} \text{norm} &= 1 - 0.75 + 0.75 \cdot \frac{2000}{1000} \\ &= 1.75 \\ \text{TF component} &= \frac{20 \cdot 2.2}{20 + 1.2 \cdot 1.75} \\ &= \frac{44}{22.1} = 1.99 \end{aligned}$$

Observation

Same TF, but short document scores higher due to length penalty

BM25 Example: Rare vs. Common Terms

Collection: $N = 100{,}000$ documents

Term	df	IDF	Interpretation
quantum	100	6.21	Very discriminative
algorithm	5,000	2.99	Moderately discriminative
system	50,000	-0.41	Not discriminative
the	99,000	-6.21	Actively harmful

Key Insight

IDF difference drives discrimination:

- Rare terms dominate scoring
- Common terms contribute little or negatively

BM25 Parameter Tuning Guidelines

Standard Defaults

$k_1 = 1.2$, $b = 0.75$ (works well for many collections)

When to adjust k_1:

- **Verbose collections** (news, articles)
  $\rightarrow$ Higher k_1 (1.5–2.0)
- **Keyword collections** (tweets, queries)
  $\rightarrow$ Lower k_1 (0.8–1.2)

When to adjust b:

- **Variable-length docs** (web pages)
  $\rightarrow$ Higher b (0.8–0.9)
- **Uniform-length docs** (abstracts)
  $\rightarrow$ Lower b (0.5–0.7)

In Lecture 6 (Learning to Rank)

You'll learn to tune these parameters systematically using labeled data

The Vocabulary Mismatch Problem

Core Issue

Queries and relevant documents use different vocabulary:

Medical Domain

- Query: "heart attack"
- Document: "myocardial infarction"

Technical Domain

- Query: "ML"
- Document: "machine learning"

General Domain

- Query: "elderly care"
- Document: "geriatric nursing"

Result

- BM25 score = 0
- Relevant doc not retrieved

Sparse Retrieval Limitation

Exact term matching only — no semantic understanding

Relevance Feedback Overview

Explicit Relevance Feedback

User marks documents as relevant/non-relevant

- ✗ Impractical: users won't give feedback
- ✗ Chicken-and-egg: need good initial results

Pseudo Relevance Feedback (PRF)

Assumption: Top-k retrieved documents are relevant

- ✓ No user interaction needed
- ✓ Automatic query expansion
- ✗ Risk: If assumption fails $\rightarrow$ query drift

Process:

Rocchio Algorithm

Historical Context

Classic PRF method from 1971, vector space approach

Formula

$$\vec{q}_{\text{new}} = \alpha \vec{q}_{\text{original}} + \beta \frac{1}{|R|} \sum_{d \in R} \vec{d} - \gamma \frac{1}{|NR|} \sum_{d \in NR} \vec{d}$$

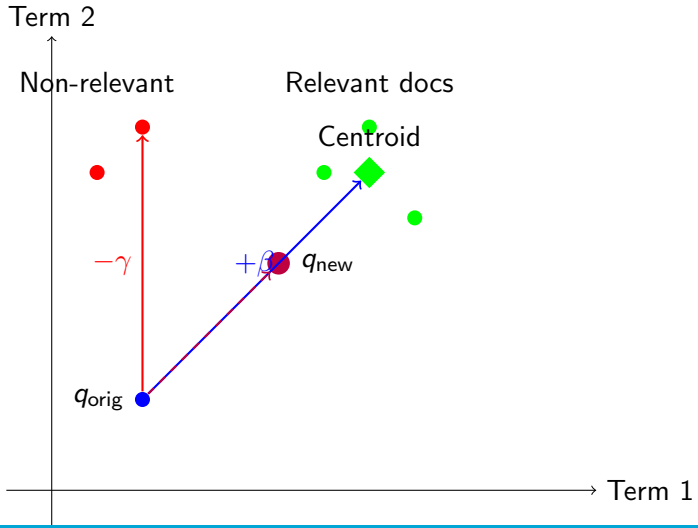
Parameters:

- $\alpha = 1$: Keep original query (always)
- $\beta = 0.75$: Weight toward positive examples
- $\gamma = 0.15$: Weight away from negative examples
- In PRF: $\gamma = 0$ (no negative feedback available)

Geometric Intuition:

- Move query vector toward centroid of relevant documents

Rocchio: Geometric Intuition



Rocchio Limitations

✗ Why Not Standard Anymore?

1. **Requires vector space representation**
   - All terms contribute (even noisy ones)
   - No principled term selection
2. **No probabilistic foundation**
   - Weights based on geometry, not probability
   - Harder to interpret
3. **Empirically outperformed**
   - RM3 achieves better results
   - Probabilistic methods more robust

Modern Usage

Mainly historical importance — good to know the name and concept, but RM3 is the standard

RM3: Relevance Model 3

Foundation

Probabilistic query expansion using language modeling framework

Two-stage process:

1. **Estimate Relevance Model**

$$P(w|R) \propto \sum_{d \in R} P(w|d) \cdot P(d|q)$$

   - For each word w, estimate probability from "relevance"
   - Weight by how relevant each doc appears: $P(d|q)$ from initial retrieval
   - Select top `fb_terms` words by $P(w|R)$

2. **Interpolate with Original Query**

$$P(w|Q_{\text{new}}) = (1 - \lambda) \cdot P(w|Q_{\text{original}}) + \lambda \cdot P(w|R)$$

RM3: Key Parameters

fb_docs (Feedback Documents): 3–10

How many top documents to use for expansion

- **Too few:** May miss relevant terms
- **Too many:** Risk including non-relevant documents
- **Standard:** 10

fb_terms (Feedback Terms): 10–20

How many expansion terms to add

- **Too few:** Limited expansion benefit
- **Too many:** Noise from marginally relevant terms
- **Standard:** 10

λ (Interpolation Weight): 0.5–0.8

RM3 Example

Original Query: "machine learning"

Top-3 Retrieved Documents (fb_docs=3):

1. "Neural networks and deep learning algorithms..."
2. "Supervised machine learning techniques..."
3. "Machine learning applications in AI..."

Extracted Terms (fb_terms=5):

- neural (P=0.15)
- deep (P=0.12)
- supervised (P=0.10)
- algorithms (P=0.09)
- AI (P=0.08)

Expanded Query ($\lambda = 0.6$):

Rocchio vs. RM3

Aspect	Rocchio	RM3
Foundation	Geometric (vector space)	Probabilistic (LM)
Term selection	All terms (weighted)	Top-k by probability
Weighting	Vector magnitude	$P(w\|R)$
Interpretability	Geometric intuition	Probabilistic
Performance	Good	Better
Usage today	Historical	Standard

Why RM3 Wins

- Probabilistic foundation (principled)
- Selective term addition (only strong associations)
- Better empirical performance
- Standard in modern systems (Indri, Anserini, Pyterrier)

When Query Expansion Helps vs. Hurts

✓ Helps When:

- Initial results mostly relevant
- Vocabulary mismatch present
- Broad informational queries
- Query: "climate change"
- Expansion adds: "warming", "CO2", "emissions"

✗ Hurts When:

- Initial results off-topic
- Ambiguous queries
- Very specific queries
- Query: "jaguar"
- Expansion adds: "car", "speed" (wrong sense!)

Query Drift Example

Query: "python programming"
Top result: Article about Python snake species
Expansion: "reptile", "snake", "habitat", "venom"
Result: Completely wrong direction!

A Different Ranking Philosophy

BM25 Approach:

- "How well does document match query?"
- Scoring based on term overlap
- Weighted by importance (IDF, TF)

Language Model Approach:

- "How likely would this document generate this query?"
- Rank by $P(q|d)$
- Probabilistic generative model

Matching Score

$$\text{score}(q, d) = \sum_t w(t, q, d)$$

Generative Probability

$$\text{score}(q, d) = P(q|d)$$

Generative Story

1. Pick a document d
2. Document has a language model (probability distribution over words)

Why Language Models Matter

Principled Probabilistic Framework

- Solid theoretical foundation (compared to heuristic weighting)
- Smoothing becomes natural and essential
- Interpretable as generative process

Connection to Modern IR

- Foundation for neural language models (Lecture 4)
- BERT, GPT are sophisticated language models
- Understanding statistical LMs helps understand neural LMs

Preview

In Lecture 4, we'll see how transformers extend this probabilistic foundation with

Query Likelihood Model

Basic Formulation

Rank documents by probability they would generate the query:

$$P(q|d) = \prod_{w \in q} P(w|d)^{\text{count}(w,q)}$$

Maximum Likelihood Estimate:

$$P_{\text{ML}}(w|d) = \frac{\text{count}(w, d)}{|d|}$$

Critical Problem: Zero Probabilities

If query word doesn't appear in document:

- $P(w|d) = 0$

Zero Probability Example

Query: "machine learning algorithms"

Document: "This paper presents a novel neural network approach to supervised machine learning."

Term	**count(w,d)**	$P_{\mathbf{ML}}(w\|d)$	**Result**
machine	1	$1/15 = 0.067$	✓
learning	1	$1/15 = 0.067$	✓
algorithms	0	$0/15 = 0$	✗ ZERO!
$P(q\|d)$	$0.067 \times 0.067 \times 0 = \mathbf{0}$		

Problem

Document is clearly relevant, but one missing term kills the entire score!

Smoothing: Core Idea

Principle

Mix document evidence with collection evidence:

$$P(w|d) = f(\text{document evidence}, \text{collection evidence})$$

Collection Language Model:

$$P(w|C) = \frac{\sum_{d' \in C} \text{count}(w, d')}{\sum_{d' \in C} |d'|}$$

Two Main Approaches:

1. **Jelinek-Mercer:** Fixed linear interpolation
2. **Dirichlet:** Length-adaptive interpolation

Effect

Jelinek-Mercer Smoothing

Formula

$$P(w|d) = (1 - \lambda) \cdot P_{\text{ML}}(w|d) + \lambda \cdot P(w|C)$$

Parameter λ (typically 0.1–0.3):

- How much to trust collection vs. document
- $\lambda = 0$: Only use document (no smoothing)
- $\lambda = 1$: Only use collection (ignore document)
- $\lambda = 0.1$: Mostly trust document, small collection backup

Properties:

- ✓ Simple and interpretable
- ✓ Same λ for all documents
- ✗ Doesn't adapt to document length

Dirichlet Smoothing

Formula

$$P(w|d) = \frac{\text{count}(w, d) + \mu \cdot P(w|C)}{|d| + \mu}$$

Parameter μ (typically 1000–2500):

- Smoothing strength (number of "pseudo-counts" from collection)
- Higher μ: More smoothing (trust collection more)
- Lower μ: Less smoothing (trust document more)

Key Difference: Length-Adaptive

- **Short documents:** μ dominates denominator $\rightarrow$ rely more on collection
- **Long documents:** Actual counts dominate $\rightarrow$ rely on document
- Automatically adapts smoothing to document length!

Dirichlet Smoothing: Length Adaptation Example

Term "algorithm" with $P(w|C) = 0.001$**,** $\mu = 2000$

Document	**count(w,d)**	$\|d\|$	$P(w\|d)$
Short doc	2	100	$\frac{2+2000\times0.001}{100+2000} = \frac{4}{2100} = 0.0019$
Long doc	2	2000	$\frac{2+2000\times0.001}{2000+2000} = \frac{4}{4000} = 0.001$

Observation

- Same term count (2 occurrences)
- Short doc gets higher probability (less smoothing needed)
- Long doc gets more smoothing (closer to collection probability)
- This is **automatic** based on $|d|$

Jelinek-Mercer vs. Dirichlet

Aspect	Jelinek-Mercer	Dirichlet
Formula	$(1-\lambda)P_{\text{ML}}(w\|d) + \lambda P(w\|C)$	$\frac{\text{count}(w,d)+\mu P(w\|C)}{\|d\|+\mu}$
Parameter	λ (0.1–0.3)	μ (1000–2500)
Length adaptation	Fixed for all docs	Automatic
Smoothing amount	Same for all docs	More for short docs
Interpretation	Linear interpolation	Pseudo-counts
Best for	Uniform-length collections	Variable-length collections

Which to Use?

- **Dirichlet generally better** for most collections
- Standard in Indri, Galago search engines
- But requires tuning μ

Language Models vs. BM25

Aspect	BM25	Language Models
Philosophy	Term matching score	Generative probability
TF handling	Explicit saturation (k_1)	Implicit via smoothing
Length norm	Explicit (b)	Implicit via smoothing
IDF	Explicit component	Through $P(w\|C)$
Zero probabilities	Avoided via IDF	Smoothing essential
Parameters	k_1, b	λ or μ
Foundation	Probabilistic relevance	Language modeling

Practical Comparison

- With appropriate parameters, rankings often similar
- BM25: Better understood, more robust defaults
- Dirichlet LM: Better for variable-length collections

Connection: RM3 Uses Language Modeling

Recall RM3 Formula

$$P(w|R) \propto \sum_{d \in R} P(w|d) \cdot P(d|q)$$

Key Insight:

- RM3 uses language modeling framework
- $P(w|d)$ estimated for each pseudo-relevant doc
- Expansion terms selected by $P(w|R)$
- RM3 builds a *language model of relevance*

Why This Matters

Understanding language models helps you understand query expansion:

- Both use probabilistic term distributions

PyTerrier Setup

```python
import pyterrier as pt
if not pt.started():
    pt.init()

# Load dataset (VASWANI: small, fast for teaching)
dataset = pt.get_dataset('irds:vaswani')

# Index (or use pre-built)
if not os.path.exists("./vaswani_index"):
    indexer = pt.IterDictIndexer("./vaswani_index")
    index_ref = indexer.index(dataset.get_corpus_iter())
else:
    index_ref = pt.IndexRef.of("./vaswani_index")

index = pt.IndexFactory.of(index_ref)

# Get topics and qrels
topics = dataset.get_topics().head(10)  # Subset for speed
```

Exercise 1: BM25 Baseline

```
# Default BM25 (k1=1.2, b=0.75)
bm25 = pt.BatchRetrieve(index, wmodel="BM25")

# Retrieve results
results = bm25.transform(topics)

# Examine one query
print(results[results['qid']=='1'][['qid', 'docno', 'score', 'rank']].
    head())
```

Output:

qid	docno	score	rank
1	51	8.234	1
1	486	7.921	2
1	12	7.456	3

Exercise 1: Parameter Effects

```python
# Create BM25 variants
bm25_default = pt.BatchRetrieve(index, wmodel="BM25")
bm25_high_k1 = pt.BatchRetrieve(index, wmodel="BM25",
                                controls={"bm25.k1": "2.0"})
bm25_low_b = pt.BatchRetrieve(index, wmodel="BM25",
                              controls={"bm25.b": "0.3"})
bm25_high_b = pt.BatchRetrieve(index, wmodel="BM25",
                               controls={"bm25.b": "0.95"})

# Compare
comparison = pt.Experiment(
    [bm25_default, bm25_high_k1, bm25_low_b, bm25_high_b],
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "P_10"],
    names=["Default", "k1=2.0", "b=0.3", "b=0.95"]
)
print(comparison)
```

Exercise 2: RM3 Query Expansion

```python
# RM3 pipeline: retrieve -> expand -> retrieve again
bm25 = pt.BatchRetrieve(index, wmodel="BM25")
rm3 = pt.rewrite.RM3(index)  # Default: fb_docs=10, fb_terms=10
pipeline_rm3 = bm25 >> rm3 >> bm25

# Compare with baseline
comparison = pt.Experiment(
    [bm25, pipeline_rm3],
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "recall_100"],
    names=["BM25", "BM25+RM3"]
)
print(comparison)

# Inspect expanded query
expanded = rm3.transform(topics)
print("Original:", topics[topics['qid']=='1']['query'].values[0])
print("Expanded:", expanded[expanded['qid']=='1']['query'].values[0])
```

Exercise 2: RM3 Parameter Tuning

```
# Try different RM3 configurations
rm3_configs = {
    "RM3 (default)": pt.rewrite.RM3(index),
    "RM3 (5 docs)": pt.rewrite.RM3(index, fb_docs=5),
    "RM3 (15 docs)": pt.rewrite.RM3(index, fb_docs=15),
    "RM3 (5 terms)": pt.rewrite.RM3(index, fb_terms=5),
    "RM3 (30 terms)": pt.rewrite.RM3(index, fb_terms=30),
}

pipelines = {name: bm25 >> rm3_cfg >> bm25
             for name, rm3_cfg in rm3_configs.items()}

results = pt.Experiment(
    [bm25] + list(pipelines.values()),
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10"],
    names=["BM25"] + list(pipelines.keys())
)
```

Exercise 3: Language Models

```
# Dirichlet LM with different mu values
lm_1000 = pt.BatchRetrieve(index, wmodel="DirichletLM",
                           controls={"c": "1000"})
lm_2500 = pt.BatchRetrieve(index, wmodel="DirichletLM",
                           controls={"c": "2500"})  # default
lm_5000 = pt.BatchRetrieve(index, wmodel="DirichletLM",
                           controls={"c": "5000"})

# Compare with BM25
comparison = pt.Experiment(
    [bm25, lm_1000, lm_2500, lm_5000],
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "P_10"],
    names=["BM25", "LM (mu=1000)", "LM (mu=2500)", "LM (mu=5000)"]
)
print(comparison)
```

Exercise 4: Comprehensive Comparison

```python
# Build all systems
systems = {
    "BM25": bm25,
    "Dirichlet LM": lm_2500,
    "BM25 + RM3": bm25 >> pt.rewrite.RM3(index) >> bm25,
    "LM + RM3": lm_2500 >> pt.rewrite.RM3(index) >> lm_2500,
}

# Final evaluation
final_results = pt.Experiment(
    list(systems.values()),
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "P_10", "recall_100"],
    names=list(systems.keys()
)

print(final_results)
```

Hands-On: Student Exercises

Exercise 1: BM25 Parameters (5 mins)

- Pick one query
- Compare scores for top document across different b values
- Find a long document and short document
- How does b affect their relative scores?

Exercise 2: Query Expansion Analysis (7 mins)

- Find a query where RM3 helps significantly
- Find a query where RM3 hurts (query drift)
- Look at expanded terms — are they semantically related?
- What happens with fb_docs=3 vs. fb_docs=20?

Exercise 3: BM25 vs. LM (5 mins)

What We've Covered Today

Sparse, Term-Based Ranking

- **BM25:** Probabilistic relevance, saturating TF, length normalization
- **Language Models:** Generative probabilities, smoothing essential
- **Query Expansion:** RM3 for vocabulary mismatch

Fundamental Limitations

- Still bag-of-words (order doesn't matter)
- Exact term matching only
- No semantic understanding
- "car" and "automobile" completely different
- "bank" (river) vs. "bank" (financial) treated identically

Preview: Lecture 4 — Transformer Re-rankers

Why Transformers?

Sparse methods don't understand semantics

- Synonyms not recognized
- Polysemy not resolved
- Context not considered

Solution: Contextualized Embeddings

- BERT: Bidirectional transformer language model
- Each word embedding depends on context
- "bank" near river $\rightarrow$ different embedding than "bank" near money

Re-ranking Architecture

1. BM25/LM retrieves 1000 candidates (fast, today's lecture)

Preview: Lecture 5 — Dense Retrieval

Why Dense Retrieval?

Query expansion helps but doesn't eliminate vocabulary mismatch

- RM3 adds related terms, but still exact matching
- "elderly care" and "geriatric nursing" still won't match

Solution: Semantic Embeddings

- Embed queries and documents in dense vector space
- Similarity measured by dot product or cosine
- Semantically similar items close in embedding space
- "elderly care" $\approx$ "geriatric nursing" in embedding space

✗ Trade-off

- ✓ No vocabulary gap

Preview: Lecture 6 — Learning to Rank

Why Learning to Rank?

We've been hand-tuning parameters:

- BM25: k_1, b
- Language models: μ or λ
- RM3: fb_docs, fb_terms

What if we could learn optimal settings from data?

Solution: Supervised Learning

1. Extract features (BM25 score, LM score, doc length, etc.)
2. Train model to optimize ranking directly
3. Common approaches: LambdaMART, RankSVM, LambdaRank

Feature Engineering

The Modern Retrieval Stack

Stage 1: BM25/LM retrieve 1000 candidates
(Today's lecture)

↓

Stage 2: RM3 improves recall
(Today's lecture)

↓

Stage 3: Transformer re-ranks top 100
(Lecture 4)

↓

Alternative: Dense retrieval replaces Stage 1
(Lecture 5)

↓

Final: LTR optimizes entire pipeline
(Lecture 6)

Why Sparse Methods Still Matter

1. **Efficiency**
   - Orders of magnitude faster than dense retrieval
   - BM25: microseconds, Dense: milliseconds
   - Essential for large-scale systems
2. **Transparency**
   - Explainable scores (IDF, TF contributions)
   - Debuggable (inspect why doc ranked high)
   - Neural models are black boxes
3. **Hybrid Systems**
   - Best results combine sparse + dense
   - Sparse catches exact matches, dense catches semantic
   - Complementary strengths
4. **First-Stage Retrieval**
   - Neural methods too slow for full corpus
   - BM25 narrows down candidates
   - Neural re-ranking on top
5. **Feature Engineering for LTR**

Key Takeaways

1. BM25 is the Industry Standard

- Probabilistic foundation, robust performance
- Tune k_1 for TF saturation, b for length normalization
- Defaults ($k_1 = 1.2$, $b = 0.75$) work well for most collections

2. RM3 Improves Recall

- Addresses vocabulary mismatch via query expansion
- Watch for query drift (test on validation set)
- Typically improves MAP by 5-15%

3. Language Models Provide Principled Alternative

- Generative probabilistic framework
- Smoothing is essential (handles zero probabilities)

Assignment: Sparse Ranking Comparison

Due: Before Lecture 5

Part 1: Baseline Comparison

- Implement BM25 and Dirichlet LM on new collection
- Tune parameters: 3+ values each for k_1, b, μ
- Report MAP, NDCG@10, P@10

Part 2: Query Expansion

- Add RM3 to your best baseline
- Experiment with fb_docs (3, 5, 10, 15) and fb_terms (5, 10, 20)
- Identify queries helped vs. hurt by expansion

Part 3: Analysis

- For 3 queries, explain ranking differences between BM25 and LM

Assignment: Looking Ahead

Part 4: Connecting to Future Lectures

Based on your experimental results:

- Which queries would benefit most from semantic matching (dense retrieval)?
- Which might still need sparse retrieval?
- What features would you extract for learning to rank?

Deliverable

Jupyter notebook with:

- Code for all experiments
- Results table with all metrics
- Written analysis (2-3 pages)
- Bring your best BM25/LM pipeline to Lecture 4

Next Class: Transformer Re-rankers

Preparation

Read:

- Vaswani et al. (2017): "Attention Is All You Need"
- Devlin et al. (2019): "BERT: Pre-training of Deep Bidirectional Transformers"

Skim:

- Nogueira & Cho (2019): "Passage Re-ranking with BERT"

What We'll Build

- Use your BM25 pipeline as first-stage retriever
- Add BERT re-ranker on top
- Compare sparse vs. neural ranking
- Understand attention mechanisms

Questions?

Questions?

Office Hours: [Your time]

Email: [Your email]

Good luck with the assignment!

Backup: BM25 Variants

BM25+

Adds query term frequency normalization:

$$\text{BM25+}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \left(\frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \text{norm}} + \delta \right)$$

- δ: Lower bound on term contribution
- Not widely adopted in practice

BM25F (Fielded Documents)

Extends BM25 for documents with multiple fields (title, body, etc.):

- Different k_1, b per field
- Field weights

Backup: Other Smoothing Methods

Absolute Discounting

$$P(w|d) = \frac{\max(\text{count}(w,d) - \delta, 0)}{|d|} + \frac{\delta \cdot |V_d|}{|d|} \cdot P(w|C)$$

- Subtracts fixed discount δ from each count
- Redistributes to unseen words
- Less common than JM or Dirichlet

Two-Stage Smoothing

Combine multiple smoothing methods:

- Stage 1: Document-level smoothing (Dirichlet)
- Stage 2: Collection-level smoothing (JM)
- Can improve performance but more complex

Backup: PyTerrier Advanced Features

Custom Weighting Models

```python
# Implement your own weighting model
class MyWeightingModel(pt.Transformer):
    def transform(self, topics):
        # Custom scoring logic
        return results
```

Feature Extraction for LTR

```python
# Extract multiple features
features = pt.FeaturesBatchRetrieve(
    index,
    wmodel="BM25",
    features=["WMODEL:BM25", "WMODEL:DirichletLM", "WMODEL:TF_IDF"]
)
```